ITDS211 · หน้า 1/4 · Ch01–03 Basic · Bus · Interrupt · Data Rep

★ อาจารย์เน้น · ⚠ ข้อหลอก · † ผมเสริมเอง (ไม่มีในสไลด์/ซับ)

Ch01 พื้นฐาน — Basic Computer Components

- **Architecture** = สิ่งที่ programmer เห็น (ISA, addressing mode, data type) "อะไร" · **Organization** = ตัว implement (cache, pipeline, bus width, clock) "อย่างไร"
- องค์ประกอบ: CPU (**Control Unit** · **ALU** · **Registers** · interconnect), Memory, Storage, I/O, Motherboard, PSU, GPU
- ★ **Machine cycle**: Fetch → Decode → Execute → Store (Write back) — อาจารย์ย้ำเรื่อง **ลำดับ**
- **PC** = address ของคำสั่งถัดไป (+ หลัง fetch) · **IR** = คำสั่งที่ fetch มา · **Status/Flag** = Overflow, Negative
- **Multi-core** = hardware จริง ทำงานขนาน · **Multi-thread** = แบ่งงานเสมือนบน core (Hyper-Threading / SMT)
- **RISC** = Reduced Instruction Set (Decoder ถอดตรง) · **CISC** = Complex (microprogram mPC/mPM) · pipeline ใช้ได้ทั้งคู่
- Cloud: **SaaS** ให้ซอฟต์แวร์ · **PaaS** ให้แพลตฟอร์มรัน app · **IaaS** ให้ VM/ฮาร์ดแวร์ผ่าน API
- 8 Great Ideas: Moore's Law · Abstraction · Common case fast · Parallelism · Pipelining · Prediction · Hierarchy of memories · Dependability via redundancy

Ch01 อื่น ๆ ในสไลด์ — Below your program · Classes · Patterns

- **Below your program**: Application software (เขียนด้วย high-level) → **System software** (**Compiler** แปล HLL → machine code · **OS** จัดการ I/O, memory/storage, scheduling) → Hardware (processor, memory, I/O controller) · High-level → Compiler → **Assembly** → Assembler → **Binary**
- **Supercomputer** = คำนวณวิทย์/วิศวกรรมระดับสูงสุด ส่วนแบ่งตลาดเล็ก · **Embedded** = ซ่อนเป็นส่วนประกอบในระบบ (หลายพันล้านชิ้น/ปี) ผูกกับสภาพแวดล้อม มี **real-time** ทำหลายกิจกรรมพร้อมกัน ข้อจำกัด power/performance/cost เข้ม · **ARM Cortex-M** = ตอบ interrupt เร็วและ deterministic กินไฟต่ำ
- Parallelism ในแอป: **DLP** (data-level) · **TLP** (task-level) · ในสถาปัตยกรรม: ILP · Vector/GPU · Thread-level · Request-level
- System pattern: **Monolithic** (ระบบเดียว) · **Client-server** (แยกฝั่งขอ/ฝั่งให้บริการ) · **Distributed** (กระจายหลายเครื่อง) · **Microservices** (บริการย่อย deploy แยกได้)

Ch01 Performance · CPU time · CPI — ★ ข้อคำนวณ

Performance = 1 / Execution time
X เร็วกว่า Y n เท่า: $Perf_X/Perf_Y = Time_Y/Time_X = n$ **(ตย. A 10 s, B 15 s → 15/10 = 1.5)**

CPU time = Clock cycles × Cycle time = Clock cycles / Clock rate
Clock cycles = IC × CPI ⇒ **CPU time = IC × CPI × Cycle time = IC × CPI / Clock rate**

- Clock rate = 1/period (250 ps = 0.25 ns · 4 GHz = 4×10^9 Hz) · Elapsed time (รวม I/O, OS, idle) ≠ CPU time (user + system)
- IC ← โปรแกรม/ISA/compiler · CPI ← hardware · CPI เฉลี่ย ← instruction mix († = Σ $CPI_i \times IC_i/IC$)

ตย. CPI (สไลด์) ISA เดียวกัน A: 250 ps, CPI 2.0 · B: 500 ps, CPI 1.2 → A = I×2.0×250 = **500I** ps · B = I×1.2×500 = **600I** ps → **A เร็วกว่า 1.2 เท่า** (IC ตัดทิ้ง ถ้าไม่รู้ค่าติดตัวแปรไว้)

ตย. Clock (สไลด์) A: 2 GHz, 10 s · B ต้องเสร็จใน 6 s แต่ใช้ cycles มากกว่า 1.2 เท่า → $Clock\ rate_B = 1.2\times(10\times2\times10^9)/6$ = **4 GHz**

⚠ "เร็วกว่า" = ใช้เวลา**น้อยกว่า** · อ่านโจทย์ดี ๆ ว่าใครหารใคร (บัญญัติไตรยางศ์)

Ch02 Model · Bus · Arbitration — ★ Bus Arbitration 4 แบบ

Von Neumann	Harvard
Memory **เดียว** เก็บคำสั่ง+ข้อมูล (stored program) · เข้าถึงแบบ sequential	Instruction memory **แยก** Data memory (+bus แยก) ตัดคอขวด แต่ซับซ้อน · DSP/Microcontroller · Hybrid = Harvard ที่ L1, Von Neumann ที่ main memory

- Bus: **Address** (ตำแหน่ง Memory/I/O) · **Data** (ข้อมูล) · **Control** (ทิศทาง/ใครใช้ bus ได้เมื่อไร, เปิด-ปิด)
- **Point-to-point** = แยกทุกเส้น ง่ายแต่**เปลืองสาย** · **Multipoint** = shared ประหยัด (ไฮเวย์+ซอย) แต่ต้องมี **protocol**

Arbitration	หลักการ
Daisy chain	ส่งสิทธิ์ตาม **priority** สูง→ต่ำ (CPU สูงสุด) นิยมสุด · ข้อเสีย: priority ต่ำ (I/O) แทบไม่ได้ใช้ bus
Centralized parallel	ทุกอุปกรณ์ต่อตรงเข้า **arbitration circuit กลาง**
Distributed self-detection	ไม่มีตัวกลาง อุปกรณ์**ตัดสินกันเอง** เช็กว่ามีใครส่งอยู่ไหม
Distributed collision-detection	ส่งได้เลย ถ้าข้อมูล**ชน**เสียหาย → **ส่งซ้ำ** (retransmit)

⚠ ตัวที่เลือกว่าใครได้ใช้ bus = **Bus Arbitrator / arbitration circuit** (ไม่ใช่ Control Unit · ไม่ใช่ control line)

- **Hardware** (วงจรเฉพาะงาน เร็ว แต่ตายตัว) vs **Software** (instruction codes + interpreter ยืดหยุ่น = ร้านตามสั่ง)

Ch02 Register · Instruction cycle — Program Execution

- **MAR** = address ใน memory ที่จะอ่าน/เขียน · **MBR** = data ที่อ่านมา/จะเขียน · **IOAR** = address อุปกรณ์ I/O · **IOBR** = data แลกกับ I/O · **AC** = accumulator
- **State diagram**: Instruction address calc → **Instruction fetch** → Instruction operation **decoding** → Operand address calc → **Operand fetch** (หลาย operand วนได้) → **Data operation** → Operand address calc → **Operand store** (หลาย result วนได้) → **Interrupt check** (ไม่มี → คำสั่งถัดไป · มี → ทำ ISR)

ตย. สไลด์ คำสั้ง 16 บิต = **opcode 4 บิต + address 12 บิต** · `0001` Load AC←M · `0010` Store AC→M · `0101` Add M ใส่ AC
Mem: 300: `1940` 301: `5941` 302: `2941` · 940: `0003` 941: `0002`
PC=300 → IR=1940 (Load, addr 940) → AC=0003 → PC=301, IR=5941 → AC=3+2=**0005** → PC=302, IR=2941 → M[941]=0005

Ch02 Interrupt — ★ ขั้นตอน "จำไว้เลย"

- Interrupt = ขัดจังหวะการทำงานแบบ sequence ของ CPU · ชนิด: **Program** (overflow, หาร 0, illegal instr) · **Timer** · **I/O** (เสร็จ/ขอบริการ/error) · **Hardware failure** (ไฟดับ, parity) · Hardware vs Software interrupt (system call)
  1. **เซฟ** context (PC/PSW, register, ตัวแปร) ไว้ก่อน
  2. ดู **Interrupt Vector table** หา **ISR address** (ใครเป็นคนเรียก)
  3. ใส่ address นั้นใน **PC** → กระโดดไปทำ **ISR** (Interrupt Service Routine)
  4. เสร็จ → **Restore** ค่าที่เซฟ แล้วกลับไปทำงานเดิมต่อ
- **Sequential**: ทำทีละตัวตามลำดับ (เกิด→ทำ→กลับ→เกิดอีก) · **Nested**: ระหว่าง ISR ของ X มี Y ซ้อน → เซฟสถานะ X → ทำ Y → คืน X → ทำ X ต่อ → return · **ซ้อนได้กี่ชั้นขึ้นกับ CPU** (จำนวนชุด register ที่เก็บ context)
- มักปิด interrupt ระหว่างทำ ISR (**maskable** ปิดได้ · **nonmaskable** ต้องทำ) · interrupt ต้องรอ **instruction cycle จบ**

Ch03 Data Representation — ★ "จะมีข้อนี้แน่ๆ" ต้องทำได้เต็ม

	Sign-Magnitude	1's complement	2's complement
ค่าลบ	กลับ **sign bit** (0=+, 1=−) ที่เหลือ = magnitude	กลับ**ทุกบิต** = $(2^n-1)-m$	กลับทุกบิต **+1** = 2^n-m
ช่วง 8 บิต	−127…+127	−127…+127	**−128…+127**
ศูนย์	2 ตัว 0000 0000 / 1000 0000	2 ตัว 0000 0000 / 1111 1111	**ตัวเดียว** (ช่วงไม่สมมาตร)
บวก/ลบ	บวกเลขลบ ≠ ลบ ต้องวงจรต่างหาก	มี carry-out → **บวกกลับ LSB**	ไม่ต้อง end-around · ลบ = บวก 2's แล้ว**ทิ้ง carry-out** · ไม่ต้องวงจรลบ

- n บิต: unsigned 0…2^n-1 · 2's: $-2^{n-1}…2^{n-1}-1$ · sign extension: ขยายด้วยบิต sign (lb) / zero-extend (lbu) · สูตร r's comp = r^n-m ; $(r-1)$'s = $(r^n-1)-m$ (ตย. m=382, n=3: 9's = **617**, 10's = **618**)
- **Shortcut 2's**: จาก LSB หา **1 ตัวแรก** คงไว้ แล้วกลับบิตทางซ้ายทั้งหมด · ตย. 0101011 → 1010101
- Sign-Magnitude บวก/ลบ †: sign เหมือนกัน → บวก magnitude (carry-out = overflow) · sign ต่าง → ลบตัวเล็กจากตัวใหญ่ ใช้ sign ตัวใหญ่ · A−B = A+(−B) กลับ sign B
- Overflow (2's) †: (+)+(+) ได้ลบ หรือ (−)+(−) ได้บวก · ตย. 0111 1111 + 1 = 1000 0000
- **Bias (excess-B)**: ค่า = unsigned − B · bias 127: 0000 0000 = −127 · 0111 1111 = **0** · 1111 1111 = +128

ตย. สไลด์ 2's: 6 + (−5) 4 บิต = 0110 + 1011 = 1|**0001** (ทิ้ง carry) · 1's: 6 + (−3) 3 บิต = 110 + 100 = 1|010 → +1 = **011**

ตย. 16 บิต (แนว Assignment 2) † `+100 = 0000 0000 0110 0100` · `+37 = 0000 0000 0010 0101`
−100 SM `1000 0000 0110 0100` · 2's `1111 1111 1001 1100`
−37 SM `1000 0000 0010 0101` · 2's `1111 1111 1101 1011`
2's 100−37 = `0000 0000 0110 0100 + 1111 1111 1101 1011` = 1|`0000 0000 0011 1111` → ทิ้ง carry = **63**
2's 37−100 = `0000 0000 0010 0101 + 1111 1111 1001 1100` = `1111 1111 1100 0001` → ลบ (กลับ+1 = 63) = **−63**
SM 37−100: sign ต่าง → |100|−|37| = 63 ใช้ sign ของ 100 (ลบ) = `1000 0000 0011 1111` · 100+37 = `0000 0000 1000 1001` (137)

IEEE 754 ★ $N = (-1)^S \times [1.m] \times 2^{E-127}$ $E = e + 127$ (เก็บค่า E)

S	Exponent 8	Mantissa 23 (bit 22–0)
S	Exponent 11	Mantissa 52 (bit 51–0)

- **Single 32 บิต: exponent 8 · mantissa 23 · Double 64 บิต: exponent 11 · mantissa 52** (อาจารย์ถาม) · double bias = 1023 † · E เก็บ 1–254 (0 และ 255 สงวน: ±∞, NaN) → e = −126…+127 · max ≈ $2\times2^{127} = 2^{128}$
- ตย. **17.15** = $10001.0010011_2 = 1.00010010011\times2^4$ → E = 4+127 = **131 = 1000 0011** · S = 0 · m = 0001 0010 011…
- เศษส่วน: 101.1101_2 = 4+1+0.5+0.25+0.0625 = **5.8125** · 5.8125 → 101 และ 0.8125×2 ซ้ำ (1.625→1.25→0.5→1.0) = .1101
- **Endian ★**: **Little** (RISC-V) = byte **LSB** อยู่ address ต่ำสุดของ word · **Big** = **MSB** อยู่ address ต่ำสุด · memory เรียกทีละ **byte (8 บิต)** · ASCII 128 ตัว · Unicode 32 บิต

Ch03 Assembly (RISC-V)

★ ข้อสี: รู้ว่าคำสั่ง "ทำอะไร" · self-study ไม่ออก

คำสั่ง	ทำอะไร
`add rd,rs1,rs2`	rd = rs1 + rs2 (`sub` = ลบ) · รูปทั่วไป `ADD A,B,C` = A ← B + C (2 source + 1 dest)
`addi rd,rs1,imm`	rd = rs1 + **ค่าคงที่** · `addi x22,x22,4` → x22 ← x22 + 4
`ld/lw rd,off(rs1)`	**LOAD Memory → Register** (ld 64 บิต · lw 32 · lb/lh sign-ext · lbu/lhu/lwu zero-ext)
`sd/sw rs2,off(rs1)`	**STORE Register → Memory** (sd 64 · sw 32 · sh 16 · sb 8 เก็บบิตขวาสุด)
`and or xor` (+i)	bitwise · **and** = mask (เก็บบางบิต ที่เหลือ 0) · **or** = บังคับบิตเป็น 1 · **xor** กับ 1…1 = NOT
`slli / srli rd,rs1,i`	shift ซ้าย i บิต = **×2^i** (เติม 0) · shift ขวา = **÷2^i** (unsigned)
`beq / bne rs1,rs2,L`	กระโดดไป L ถ้า rs1 **=** rs2 / **≠** rs2 (ไม่ใช่ → ทำคำสั่งถัดไป)
`blt / bge rs1,rs2,L`	ถ้า rs1 **<** rs2 / rs1 **≥** rs2 (signed) · `bltu bgeu` = unsigned (−1 < +1 แต่ unsigned 4,294,967,295 > 1)
`jal rd,L` · `jalr`	กระโดด + เก็บ address ถัดไปใน rd (เรียก procedure: `jal x1,L`) · **return** = `jalr x0,0(x1)`
`lui rd,imm20`	ใส่ imm 20 บิตที่ rd[31:12] เคลียร์ [11:0] → สร้างค่าคงที่ 32 บิต คู่ `addi`

- **Regs** 32×64 บิต (ข้อมูล 64 บิต = doubleword · 32 บิต = word): **x0 = 0 เสมอ** · x1 return addr · x2 stack ptr · x3 global ptr · x4 thread ptr · x5–7, x28–31 temporaries · x8 frame ptr · x9, x18–27 saved · x10–11 args/ผลลัพธ์ · x12–17 args
- Memory **byte-addressed** · RISC-V = Little-endian · ไม่บังคับ word-aligned · offset = index × 8 (doubleword)

R	funct7	rs2	rs1	f3	rd	opcode
I	immediate[11:0]		rs1	f3	rd	opcode
S	imm[11:5]	rs2	rs1	f3	imm[4:0]	opcode
SB	imm[12|10:5]	rs2	rs1	f3	imm[4:1|11]	opcode
UJ	imm[20|10:1|11|19:12]				rd	opcode
Shift	funct6	immed	rs1	f3	rd	opcode

- ทุกคำสั่ง **32 บิต** · opcode **7 บิต** (2^7 = 128 แบบ) · rs1/rs2/rd **5 บิต** (2^5 = 32 reg) · funct3/funct7 = opcode เสริม · **I**: imm 12 บิต (2's, sign-ext) ใช้กับ addi/load · **S**: store แยก imm เพื่อให้ rs1/rs2 อยู่ที่เดิม · **SB**: PC-relative, **target = PC + imm × 2**

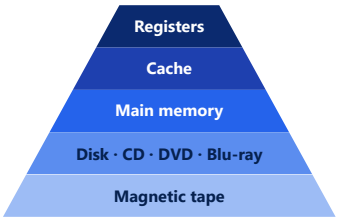
ตย. R-format `add x9,x20,x21` → funct7 **0** · rs2 **21** · rs1 **20** · f3 **0** · rd **9** · opcode **51**
`0000000 10101 10100 000 01001 0110011` = **015A04B3**$_{16}$ (ข้อสอบ: จะมี chart ให้ว่า opcode คืออะไร)

ตย. C → RISC-V (f…j = x19…x23) `f=(g+h)-(i+j)` : `add x5,x20,x21 ; add x6,x22,x23 ; sub x19,x5,x6`
`A[12]=h+A[8]` (h=x21, A=x22): `ld x9,64(x22) ; add x9,x21,x9 ; sd x9,96(x22)` (8×8=64, 12×8=96)
`if(i==j) f=g+h; else f=g-h` (i=x22, j=x23): `bne x22,x23,Else ; add x19,x20,x21 ; beq x0,x0,Exit ; Else: sub x19,x20,x21 ; Exit:`
(beq x0,x0 = กระโดดเสมอ)

- **Procedure** ★ = ฟังก์ชัน (เหมือน software interrupt): ① วาง arg ที่ x10–x17 ② `jal x1,L` ③ จองที่ + **save ค่าลง stack** (`addi sp,sp,-n` ; `sd`) ④ ทำงาน ⑤ ผลลัพธ์ใส่ x10 ⑥ **restore** จาก stack (`ld` ; `addi sp,sp,n`) ⑦ `jalr x0,0(x1)` กลับ · non-leaf ต้อง save x1 + arg/temp ที่ใช้ต่อ
- **Addressing 4 แบบ**: Immediate · Register · Base (reg + offset) · PC-relative · Memory layout: **Text** (โค้ด) · **Static data** (global, x3) · **Heap** (dynamic) · **Stack** · Design: ① Simplicity favours regularity ② Smaller is faster ③ Good design demands good compromises ④ Make the common case fast

Ch04 Memory Hierarchy

★ "ให้จำรูปพีระมิดเอาไว้"



เร็วสุด · แพงสุด · จุ๊อยสุด (ใน CPU)
SRAM 0.5–2.5 ns · $2000–5000/GB
DRAM 50–70 ns · $20–75/GB
Magnetic disk 5–20 ms · $0.20–2/GB
ช้าสุด · ถูกสุด · จุมากสุด (off-line)

- **Inboard memory** = Register + Cache + Main memory · **Outboard storage** = disk/optical · **Off-line** = tape (big data center) · **Cache**: L1 ชิด CPU ก้อนเดียวกัน, L2/L3 อยู่นอก core · **บน** = เร็ว/แพง/จุ๊อย · **ล่าง** = ช้า/ถูก/จุมาก · SRAM เร็วกว่า DRAM **~100 เท่า** (transistor on-off ไม่กี่ตัว) แต่แพงกว่า · **DRAM** = ประจุ๊ใน capacitor ต้อง **refresh** ตลอด ช้ากว่า ถูกกว่า
- **RAM** = ชั่วคราว (volatile) · **ROM** = ถาวร ปิดไฟข้อมูลยังอยู่ · **Flash** (nonvolatile; เร็วกว่า disk 100–1000×; $/GB ระหว่าง disk กับ DRAM; สึกหรอ → wear leveling): **NOR** random access ใช้เก็บคำสั่งใน embedded · **NAND** หนาแน่น/ถูก เขียนทีละ block (USB, media)
- Locality: **Temporal** = ของที่เพิ่งใช้ มักถูกใช้ซ้ำเร็ว ๆ (loop) · **Spatial** = ของใกล้ ๆ ที่เพิ่งใช้ มักถูกใช้ตาม (array, คำสั่งต่อเนื่อง)

Ch04 Cache: Hit/Miss · Direct-Mapped

★ อาจารย์ "เน้นเรื่อง Cache"

- **Cache** = หน่วยความจำเร็วคั่นกลาง CPU↔Main memory เก็บข้อมูลที่**ใช้บ่อย/ซ้ำ** · **Block (line)** = หน่วยที่ copy · **Hit** = เจอใน cache · **Miss** = ไม่เจอ → ดึง block จาก main memory มาใส่ cache แล้วส่ง CPU (เวลา = **miss penalty**) · Hit ratio = hits/accesses · Miss ratio = 1 − hit ratio

> Direct-mapped: Cache index = (Block address) modulo (#Blocks in cache) → **ใช้ บิตล่าง เป็น index**
> Address = **[Tag | Index | Block offset]** offset = $\log_2$(bytes/block) · index = $\log_2$(#blocks) · tag = **ที่เหลือ**

- เช็ก hit/miss: ดู **Index** → เทียบ **Tag** → ดู **Valid bit (V)** = 1 (เริ่มต้น cache ว่าง V=0) · ข้อเสีย: หลาย address ชิงช่องเดียวกัน (แย่งกัน/conflict)

ตย. สไลด์ cache 8 blocks, 1 word/block (Tag = 2 บิตบน · Index = 3 บิตล่าง)

ขอ	22	26	22	26	16	3	16	18
Tag Idx	10 110	11 010	10 110	11 010	10 000	00 011	10 000	10 010
ผล	Miss	Miss	**Hit**	**Hit**	Miss	Miss	**Hit**	Miss*

*18: index 010 มี tag 11 แต่ขอ tag 10 → tag ไม่ตรง → Miss แล้ว **เขียนทับ** · ครั้งแรกของทุกตัว = Miss แน่นอน (cache ว่าง)

ตย. ★ บล็อกใหญ่ 64 blocks × 16 B/block, address **1200**: Block address = floor(1200/16) = **75** → Block number = 75 mod 64 = **11** · fields: offset 4 บิต [3:0] · index 6 บิต [9:4] · tag 22 บิต [31:10]
16 KB, 4 word (16 B)/block, address 32 บิต: 1024 blocks → offset 4 · index 10 · tag **18** (+ valid + dirty) · bits ทั้งหมด † = #blocks × (8×bytes/block + tag + 1)
ตย. † 4 KB direct-mapped, 16 B/block, address 32 บิต → 4096/16 = **256 blocks** → index 8 · offset 4 · tag 32−8−4 = **20** · ถ้า **2-way**: sets = 128 → index 7 · tag 21

- **Block ใหญ่ขึ้น**: miss rate ลด (spatial locality) แต่ cache ขนาดคงที่ → blocks น้อยลง แย่งกัน miss rate กลับเพิ่ม · miss penalty สูงขึ้น · ใหญ่ ≠ ดีเสมอ ⚠

Ch04 Write Policy

★ ควรรู้นิยามสามคำ

- **Write-through**: เขียนทั้ง cache **และ memory** ทุกครั้ง (consistent แต่ช้า: CPI = 1 + 0.1×100 = 11) → ใช้ **write buffer** (CPU ไปต่อ · stall เมื่อ buffer เต็ม)
- **Write-back**: เขียน**เฉพาะ cache** + ตั้ง **dirty bit** · เขียนกลับ memory เมื่อ block ถูกแทนที่
- **Write allocation** (เมื่อ write miss): **allocate** = ดึง block มาก่อนเขียน · **write around** = ไม่ดึง เขียนลง memory ตรง ๆ · write-back มัก fetch block
- อาจารย์: เมื่อก่อนมีงานวิจัยแข่งกลไกเขียน แต่ปัจจุบัน transistor เล็ก (2–4 nm) เร็วขึ้น ความต่างน้อย → เน้นให้โค้ดที่ใช้บ่อยอยู่ **L1** ให้มากที่สุด

Ch04 Associativity · Replacement · Miss

★ Fully assoc = miss น้อยสุด

ชนิด	วางได้ที่ไหน	เทียบ tag
Direct-mapped	ที่เดียว (index) — 1 ทางเลือก	1
n-way set associative	set = (block no.) mod (#sets) · วางได้ **n ช่อง**ใน set (2-way เก็บ 2 · 4-way เก็บ 4 …)	n
Fully associative	ช่องไหนก็ได้ · เทียบทุก entry	#entries

- Set-associative = "ไม่ได้ฉลาดขึ้น" แค่**ขยายที่เก็บใน index เดียวกัน** → hit มากขึ้น · miss rate (64 KB, SPEC2000): **1-way 10.3% · 2-way 8.6% · 4-way 8.3% · 8-way 8.1%** → ดีขึ้นแต่ได้น้อยลงเรื่อย ๆ (diminishing returns)
- ตย. 4-block cache ขอ 0, 8, 0, 6, 8: Direct = M M M M M · 2-way = M M **H** M M · Fully = M M **H** M **H**
- Replacement: ถ้ามีช่อง non-valid ใช้ก่อน ไม่งั้น **LRU** (ไม่ถูกใช้นานสุด) / FIFO / LFU / **Random** (direct ไม่มีตัวเลือก)
- Miss 3 แบบ: **Compulsory** (ครั้งแรก) · **Capacity** (cache เล็ก) · **Conflict** (ชนช่อง; fully assoc ไม่มี) · ↑ขนาด cache → ลด capacity miss · ↑associativity → ลด conflict miss · ↑block size → ลด compulsory miss แต่ penalty ↑
- Cache design: addresses (logical/physical) · size · mapping · replacement · write policy · line size · จำนวน cache (1–2 level; unified/split)

Ch04 AMAT · CPU time กับ memory stall

> AMAT = Hit time + Miss rate × Miss penalty
> CPU time = (execution cycles + **memory stall cycles**) × cycle time
> Stall cycles = (Mem accesses/Program) × Miss rate × Penalty = (Instr/Program) × (Misses/Instr) × Penalty

ตย. AMAT clock 1 ns, hit 1 cycle, penalty 20 cycles, miss 5% → 1 + 0.05×20 = **2 ns** · **ตย. CPI** base CPI 2, I-miss 2%, D-miss 4%, load/store 36%, penalty 100 → 2 + 0.02×100 + 0.36×0.04×100 = 2 + 2 + 1.44 = **5.44** → ideal เร็วกว่า 5.44/2 = **2.72 เท่า**

- Multilevel: L1 เน้น **hit time ต่ำ** · L2 เน้น **miss rate ต่ำ** · block L1 < block L2 · ตย. CPI = 1 + 0.02×20 + 0.005×400 = 3.4 (เทียบ 9 ถ้ามี L1 อย่างเดียว → 2.6 เท่า)

Ch05 I/O Module · เทคนิค 3 แบบ

★ DMA เร็วสุด / นิยมใช้ปัจจุบัน

- **External device** 3 กลุ่ม: Human readable (จอ/คีย์บอร์ด) · Machine readable (disk/sensor) · Communication · ประกอบด้วย Control logic + **Buffer** + **Transducer** (แปลง digital↔analog) · สัญญาณ Control / Status / Data
- **I/O module 5 หน้าที่**: ① Control & timing ② Processor communication (ถอดรหัส command, ส่งข้อมูล, รายงาน status, จำ address) ③ Device communication ④ **Data buffering** (memory เร็ว I/O ช้า → balance speed) ⑤ Error detection
- **I/O command 4**: **Control** (สั่งให้ทำ) · **Test** (ดู status) · **Read** (ดึงข้อมูลจากอุปกรณ์เข้า buffer) · **Write** (เอาข้อมูลจาก data bus ส่งอุปกรณ์) · สายของ module: Address · Control · Data
- **I/O addressing**: **Memory-mapped** = address space เดียวกับ memory ใช้คำสั่ง memory ได้หมด ไม่มีคำสั่ง I/O พิเศษ · **Isolated** = แยก space ต้องมี I/O/memory select line + คำสั่ง I/O เฉพาะ (จำกัด)

Flow (สไลด์) Programmed: Issue Read → Read status → Check (ไม่พร้อม = **วนกลับ**) → Read word → Write ลง memory → Done? (ยัง = วนอีก) · **Interrupt-driven**: Issue Read → **ทำงานอื่น** → รอ interrupt จึง Read status/word · **DMA**: Issue Read-block ให้ DMA → **ทำงานอื่นทั้งหมด** → DMA interrupt เมื่อเสร็จ

	1 Programmed I/O	2 Interrupt-driven	3 DMA ★
CPU ทำ	สั่งแล้ว **รอ** (วนเช็ก status) ไม่มี interrupt · ง่ายสุด	สั่งแล้ว**ไปทำงานอื่น** · I/O **interrupt** เมื่อพร้อม	ตั้งค่า block command แล้วไปทำงานอื่น · DMA interrupt ตอนจบ
ข้อมูลผ่าน	I/O → **CPU** → memory	I/O → **CPU** → memory	I/O ↔ memory **ตรง** (CPU ไม่ยุ่ง)
ข้อเสีย	เสียเวลา CPU · rate จำกัดด้วย CPU	CPU ยังผูกกับการโอน · rate จำกัดด้วย CPU	ต้องมี DMA controller · flow ฝั่ง CPU ง่ายสุด

- **ทำไม DMA เร็วสุด ★**: คุยกับ memory ได้**ทุก slot ของ processor cycle** (breakpoint ก่อน fetch / หลัง decode / fetch operand / execute / store) ผ่าน hardware bus ตรง · ส่วน **interrupt ต้องรอ instruction cycle จบ** (เช่น pipeline 4–5 stage ต้องรอครบ) · DMA module: Data count · Data register · Address register · Control logic
- DMA config 3 แบบ: single-bus **detached** DMA · single-bus **integrated DMA-I/O** · **I/O bus** · ตย. chip 8237 (HRQ/HLDA, DREQ/DACK) · Direct Cache Access (DCA/DDIO) ใช้กับ network 10–100 Gbps
- วิวัฒนาการ I/O 6 ขั้น: CPU คุมตรง → controller + programmed I/O → interrupt → DMA → **I/O channel** (module เป็น processor เฉพาะ) → **I/O processor** (มี local memory)

Ch05 Interrupt-driven · Device Identification

★ 4 แบบ

- **เมื่อเกิด interrupt**: เก็บ **PC + status register (PSW)** ลง stack → PC ← จุดเริ่ม ISR → รัน ISR → **return**: คืน PC เดิม + คืนค่าใน stack ทั้งหมด แล้วทำงานเดิมต่อ · ปัญหาออกแบบ 2 ข้อ: ใครเป็นคน interrupt? · ถ้ามีหลายตัวทำตัวไหนก่อน?

Device identification	หลักการ
1 Multiple interrupt lines	แยกเส้น interrupt คนละเส้น รู้ทันทีว่าใคร · ข้อเสีย: I/O เยอะ เส้นเยอะ
2 Software poll	กระโดดไป routine ที่**ไล่ถามทีละ I/O module** ว่าใครเรียก · ช้า
3 Daisy chain	(**hardware poll**, vectored) **priority** · interrupt acknowledge ส่งต่อเป็นลูกโซ่ · vector = address/ID ของ module
4 Bus arbitration	(vectored) module ต้อง**ขอคุม bus ก่อน** จึงยก interrupt request · ได้รับตอบแล้วส่ง **vector** ผ่านสาย data/address

- ต่อ I/O เยอะ: ตัวเลือก **8-to-1** (mux) เลือกได้ทีละ 1 · I/O **64** ตัว = 8 ชุด 8-to-1 + อีก 1 ตัวเลือกจาก 8 กลุ่ม (**master/slave** interrupt controller, ตย. 82C59A)

Ch05 I/O Standards

★ assignment: USB vs SCSI · Thunderbolt · InfiniBand

มาตรฐาน	จุดสำคัญ (ตามสไลด์)
USB	1.0: Low 1.5 / Full 12 Mbps · 2.0: **480 Mbps** · 3.0/3.1 Gen1: **5 Gbps** (SuperSpeed) · 3.1 Gen2: 10 Gbps · 3.2 Gen2×2: 20 Gbps · root host controller · tree (hierarchical) · hot-plug
SCSI	**shared bus** 16/32 อุปกรณ์ · ส่งแบบ **parallel** (bus 16 → 32 บิต) · 5 Mbps (SCSI-1) → 160 Mbps (Ultra3) · เสียความนิยมให้ USB/FireWire · ยังใช้กับ enterprise mass storage
FireWire	IEEE 1394 · ทางเลือกแทน SCSI · **daisy chain** ≤ 63 อุปกรณ์/port · hot plug · auto config · ไม่ต้อง terminator
Thunderbolt	Intel+Apple · ≤ **10 Gbps** ต่อทิศ · ≤ 10 W · รวม data+video+audio+power ในสายเดียว
InfiniBand	**high-end server** · switch-based fabric ต่อได้ราว **64,000** อุปกรณ์ · storage area network · ★ อาจารย์: ใช้กับ **GPU ↔ memory ที่ต้องการความเร็วสูงสุด**
PCI Express	high-speed bus หลายชนิดอุปกรณ์ · ★ **การ์ดจอ (GPU) ต่อเมนบอร์ดผ่าน PCIe** (ไม่ใช่ USB/FireWire/Thunderbolt)
SATA / Eth / Wi-Fi	SATA = disk ≤ 6 Gbps · Ethernet ≤ 100 Gbps (switch-based) · Wi-Fi 802.11ac ≤ 3.2 Gbps

Ch06 Hard Disk

★ storage ออกไม่เยอะ เน้น "หลักการอ่าน/เขียน"

- **เขียน**: จ่ายกระแสเข้าหัวเขียน (ขดลวด) → เกิด**สนามแม่เหล็ก** → จัดทิศอนุภาคบนผิว platter ซ้าย/ขวา = **0/1** · **อ่าน**: ขดลวดหัวอ่าน**เหนี่ยวนำ**ดูทิศ (inductive write / magnetoresistive read) · platter เคลือบวัสดุแม่เหล็ก · **1 head ต่อ 1 surface** · หัวแบบ Winchester ลอยด้วยแรงลม (aerodynamic)
- **Platter → Track** (วงแหวนหนึ่งวง) → **Sector** (**ส่วนย่อยของ track** ⚠ ไม่ใช่กลุ่มของ track) · **Cylinder** = track ตำแหน่งเดียวกันทุก platter · มี gap ระหว่าง track/sector · sector ≈ 512 B (ใหม่ 4096) + ID + ECC
- **Constant angular velocity** (หั่นเค้ก sector ขนาดไม่เท่ากัน) vs **Multiple-zone recording** (density/sector ใกล้เคียงกัน ดีกว่า · **ปัจจุบันใช้แบบนี้**)
- Seek time = เลื่อนหัวไปถึง track · Rotational delay = รอต้น sector มาถึงหัว · **Access time = seek + rotational delay** · transfer time = อ่านข้อมูลเมื่ออยู่ถูกตำแหน่ง

Disk access = queuing + seek + rotational latency + transfer + controller overhead
Rotational latency **เฉลี่ย** = ½ **รอบ** = 0.5 / (rpm/60)

ตย. สไลด์ sector 512 B, 15,000 rpm, seek เฉลี่ย 4 ms, transfer 100 MB/s, controller 0.2 ms, disk ว่าง → 4 + **2** + 0.005 + 0.2 = **6.2 ms** (rotational = 0.5/(15000/60) = 2 ms · ถ้า seek จริง 1 ms → **3.2 ms**)

Ch06 ชนิดดิสก์ · ตัวอย่างค่า HDD

สไลด์ Table 6.2

- **ชนิดดิสก์**: **fixed-head** (1 หัว/track) vs **movable-head** (1 หัว/surface) · removable vs non-removable · single/double-sided · single/multiple platter · กลไกหัว: **Contact** (floppy) · **Fixed gap** · **Aerodynamic gap** (Winchester)
- Timing ของ disk I/O: Wait for device → Wait for channel → **Seek** → **Rotational delay** → **Data transfer**
- ตัวอย่างค่า: Seagate Enterprise 6 TB 7200 rpm seek 4.16 ms 216 MB/s · Laptop 2 TB 5400 rpm seek 13 ms · **latency = ½ รอบ = 30000/rpm ms** (7200 → 4.17 · 5400 → 5.56 · 10,075 → 2.98)

Ch06 RAID 0–6

★ Redundant Array of Independent Disks

3 ลักษณะ: ① OS เห็นเป็น logical drive เดียว ② data กระจายข้าม drive (**striping**) ③ มี capacity ส่วนเกินเก็บ **parity** กู้ข้อมูลได้ · N = จำนวน data disk

L	หลักการ · จำนวนดิสก์	ข้อดี / ข้อเสีย / ใช้กับ
0	**Striping** ไม่ redundant · N	I/O สูงมาก ง่าย ไม่มี parity · แต่ **ดิสก์เดียวเสีย = ข้อมูลทั้ง array หาย** · video/image editing
1	**Mirroring** (copy ซ้ำ) · 2N	redundant 100% ไม่ต้อง rebuild · read ได้จากดิสก์ใดก็ได้ · แต่ **overhead สูงสุด** · accounting/payroll
2	Parallel access + **Hamming code** (ECC) · N+m (m∝log N)	transfer สูงมาก · แต่ ECC disk เยอะ ไม่คุ้ม · **ไม่มีใช้เชิงพาณิชย์**
3	Parallel access, parity **ระดับ bit/byte** · **ดิสก์ parity เดียว** · N+1	transfer สูงสุด · ดิสก์เสียแทบไม่กระทบ · แต่ transaction rate ≈ ดิสก์เดียว · video streaming
4	Independent access, parity ระดับ **block** · **parity disk แยก** · N+1	read สูง · แต่ **write penalty** (อ่าน 2 เขียน 2) **parity disk = bottleneck** · ไม่มีใช้
5	block parity **กระจาย (distributed) ทุกดิสก์** · N+1	read สูงสุด ไม่มี bottleneck · เขียนช้ากว่า rebuild ยาก · file/database/web server · **versatile สุด**
6	block parity **2 ชุด** (dual distributed) · N+2	ทน**ดิสก์เสีย 2 ตัวพร้อมกัน** availability สูงสุด · controller ซับซ้อน write penalty สูง · mission critical

ตย. † ดิสก์ 1 TB × 4 ตัว: RAID 0 = 4 TB · RAID 1 = 2 TB (mirror) · RAID 5 = 3 TB (เสีย 1 ดิสก์ใช้ทำ parity) · RAID 6 = 2 TB

Ch06 SSD · Optical · Tape

★ SSD ดีกว่า HDD (แต่แพง) · tape ถูกสุด

- **SSD vs HDD**: SSD **เร็วกว่า** (IOPS สูง, latency ต่ำ) · **ประหยัดไฟ** · ทน/เงียบ/เย็น · อายุยาว แต่ **แพงกว่า/GB** · ปัญหา: เขียนช้าลง (ต้องอ่านทั้ง block → ลบ → เขียนกลับ) · flash **สึกหรอ** ตามจำนวนเขียน → **wear leveling**, bad-block mgmt, cache รวมเขียน · ภายในมี **SSD controller** (addressing, buffer, ECC)
- สไลด์ Table 6.5: copy/write **200–550** vs 50–120 Mbps · ไฟ 2–3 W vs 6–7 W · ≈ $0.50 vs $0.15 /GB

Optical	ข้อมูล
CD	12 cm · CD-ROM ≈ 650–682 MB · laser **780 nm** · CD-ROM อ่านอย่างเดียว (pit บน polycarbonate) · CD-R เขียนครั้งเดียว (dye) · CD-RW เขียนซ้ำ (phase change: amorphous สะท้อนน้อย / crystalline สะท้อนดี)
DVD	laser **650 nm** · 2 หน้า 2 ชั้น สูงสุด **17 GB** · มี DVD-R/RW/RAM
Blu-ray	laser **405 nm** (blue-violet) · 1 ชั้น 1 หน้า **25 GB**

⚠ **CD vs DVD ★**: ทางกายภาพแผ่นกลมเหมือนกัน ต่างที่ **sector/pit ของ DVD เล็กกว่า → density มากกว่า → เก็บได้มากกว่า** (sector CD ใหญ่/ห่างกว่า)

- **Magnetic tape ★**: **ถูกที่สุด · จุมากสุดในพีระมิด · offline** storage (big data center) · เข้าถึง**แบบ serial** · track ขนานตามความยาว · serpentine recording · LTO-1 (2000) 200 GB → LTO-8 32 TB (compressed)

ITDS211 · หน้า 4/4 · Ch07 Pipeline · Ch08 Parallel · สูตร · เช็กข้อหลอก

★ อาจารย์เน้น · ⚠ ข้อหลอก · † ผมเสริมเอง (ไม่มีในสไลด์/ซับ)

Ch07 Pipeline

★ 3 Hazards: Structural · Data · Control

- **แนวคิด** (ซักผ้า–อบ–รีด–เก็บ): แบ่งงานเป็น stage ย่อย **เหลื่อมกันทำ** ใช้ resource คนละตัวพร้อมกัน → **throughput** ↑ แต่ **latency ต่อคำสั่งเท่าเดิม** · stage ต่างกันตามชนิด CPU (RISC-V / Arm / Intel)
- ⚠ **Pipeline ใช้ได้ทั้ง RISC (เช่น ARM) และ CISC (เช่น Intel)** ไม่ใช่เฉพาะ RISC · ลำดับ: Fetch → Decode → Execute → Write back

5 stage	IF	ID	EX	MEM	WB
	Fetch คำสั่ง	Decode + อ่าน reg	Execute / คำนวณ addr	เข้าถึง memory	เขียนผลลง reg

Instr	Fetch	Reg read	ALU	Mem	Reg write	รวม
ld	200	100	200	200	100	**800 ps**
sd	200	100	200	200	–	**700 ps**
R-format	200	100	200	–	100	**600 ps**
beq	200	100	200	–	–	**500 ps**

- **Single-cycle** T_c = **800 ps** (ตามคำสั่งช้าสุด) · **Pipelined** T_c = **200 ps** (ตาม stage ช้าสุด) · speedup อุดมคติ ≈ **จำนวน stage** (ซักผ้า 4 โหลด = 8/3.5 = 2.3 · ไม่หยุด = 2n/(0.5n+1.5) ≈ 4)
- **ตย. †** ld 3 คำสั่งติดกัน: single-cycle = 3×800 = **2400 ps** · pipelined = (5 stage + 3 − 1) × 200 = **1400 ps** (n คำสั่ง k stage = (n + k − 1) × T_c)
- ISA เหมาะ pipeline: ทุกคำสั่ง **32 บิต** (x86 = 1–17 byte) · format น้อยและสม่ำเสมอ · load/store addressing
- **Pipeline registers** (IF/ID · ID/EX · EX/MEM · MEM/WB) เก็บข้อมูลจาก stage ก่อนหน้า ไม่ให้**ถูกคำสั่งถัดไปทับ** (hardware เพิ่ม = ต้นทุน) · สรุป: pipeline เพิ่ม **throughput** ไม่ใช่ latency

Hazard	สาเหตุ	วิธีแก้
Structural	**ใช้ resource ชนกัน** (เช่น memory เดียว: fetch ชน load/store)	**Stall** (หยุด เกิด bubble จน resource ว่าง) · หรือ แยก instruction/data memory (cache)
Data	ข้อมูลยังไม่พร้อม: `add x19,x0,x1` → `sub x2,x19,x3` (sub ต้องรอ add เขียน x19 ใน WB)	① **Stall** (ง่ายสุด แต่ช้า) ② **Forwarding/Bypassing**: ส่งผลลัพธ์จาก EX ย้อนเป็น input คำสั่งถัดไปเลย ไม่รอ WB (เพิ่ม datapath) ③ **Code scheduling**: compiler/assembler จัดลำดับ ย้ายคำสั่งไม่เกี่ยวข้องมาคั่น (ไม่เพิ่ม hardware)
Control	คำสั่งก่อนหน้าเป็น **branch/jump** → คำสั่งที่ fetch ไว้ **ไม่ถูกใช้**	**Stall/bubble** จน pipeline กลับปกติ (ส่วนใหญ่นิยม เพราะ prediction ต้นทุนวงจรสูง) · **Branch prediction** (เดา: static = backward taken, forward not taken · dynamic = เก็บ history ต่อ branch; ผิด → flush) · ย้าย target adder+comparator ไป ID

- **Load-use hazard**: ผลจาก `ld` ได้ที่ MEM — forward ย้อนเวลาไม่ได้ → **stall 1 cycle** · ตย. code scheduling (a=b+e; c=b+f): 13 cycles → จัดใหม่ (ย้าย load x4 ขึ้น) = **11 cycles**
- Forwarding (รายละเอียดจากสไลด์): EX hazard เมื่อ EX/MEM.Rd = ID/EX.Rs1/Rs2 · MEM hazard เมื่อ MEM/WB.Rd = Rs1/Rs2 · เฉพาะ RegWrite และ Rd ≠ x0 · mux ForwardA/B: **00** = register file · **10** = EX/MEM · **01** = MEM/WB

Ch08 Parallel Processors

★ Flynn · SMP/Cluster · Vector · GPU

Flynn	ความหมาย	ตัวอย่าง
SISD	**1 คำสั่ง 1 ข้อมูล** (CPU ปกติ) ⚠ ไม่ใช่ประมวลผลหลายคำสั่งพร้อมกัน	Pentium 4
SIMD	**1 คำสั่ง หลายข้อมูล**พร้อมกัน: vector/array processor, **GPU**	SSE (x86), GPU
MISD	หลายคำสั่ง 1 ข้อมูล — **แทบไม่มี/ไม่มีเลยแล้ว**	–
MIMD	หลายคำสั่ง หลายข้อมูล: **Shared memory (SMP)** หรือ **Distributed memory (cluster)**	Xeon e5345

- **Multi-core** = duplicate core จริง ๆ ทำงานขนาน (≠ แบ่ง task บน resource เดียว/VM) · **Multithreading** = replicate register+PC สลับ thread เร็ว: **Fine-grain** (สลับทุก cycle) · **Coarse-grain** (สลับเมื่อ stall ยาว เช่น L2 miss; ไม่ซ่อน stall สั้น) · **SMT** (คำสั่งจากหลาย thread รันพร้อมกันบน function unit ที่ว่าง; Intel Hyper-Threading 2 thread)
- **Vector processor**: function unit pipelined ลึก · **vector register** (RISC-V: v0–v31, 32×64-element) · ลดการ fetch คำสั่ง · เหมาะ data-parallel: **multiply–accumulate** (digital filter, ภาพ, linear algebra — ตย. **DAXPY**: Y = a×X + Y) · vector = ความกว้างปรับได้ + stride ≠ multimedia SIMD (SSE/MMX 128 บิต กว้างคงที่)
- **Shared memory** (SMP: hardware ให้ address space เดียว, sync ด้วย lock; UMA/NUMA): ✓ ไม่ต้อง duplicate/ไม่ต้อง sync ข้อมูล *X* ทุก core **รุมเข้า memory ตัวเดียว** · **Distributed memory** (แต่ละตัวมี memory เอง = **cluster**, message passing): *X* sync/update ยาก ✓ scalable/ถูก/availability สูง · cluster = คอมพิวเตอร์อิสระ+OS ของตัวเอง ต่อด้วย Ethernet (loosely coupled) · Grid computing เช่น SETI@home
- ปัญหา parallel programming: **Partitioning · Coordination · Communications overhead** · SPMD = Single Program Multiple Data · topology: Bus · Ring · 2D Mesh · N-cube · Fully connected · benchmark: Linpack · SPECrate · SPLASH · NAS · PARSEC · performance model: GFLOPs, arithmetic intensity = FLOPs/byte (Roofline)

Ch08 GPU

★ อาจารย์ปิดคลาสด้วยหัวข้อนี้

- **หลักการ = SIMD**: คำสั่งเดียว (บวก/ลบ) ทำกับข้อมูลจำนวนมากพร้อมกัน (ตย. ลบ **background** ภาพ — pixel = array/vector) · **highly multithreaded** (สลับ thread ซ่อน memory latency) · พึ่ง multi-level cache น้อยกว่า CPU · graphics memory กว้าง/bandwidth สูง · **heterogeneous**: CPU ทำ sequential, GPU ทำ parallel · API: CUDA, OpenGL, DirectX
- **Nvidia Fermi**: vector-based · SIMD processor 16 lanes · SIMD instruction ทำกับ 32 element รันบน 16-wide ใน 2 cycle · **Tesla**: multiprocessor (หลาย compute unit ทำร่วมกัน) · **DGX** = train AI model (แพง) · **รันโมเดลแล้ว** ใช้การ์ดจอปกติเสียบ **PCIe** ใน PC ได้
- Multicore SIMD vs GPU: SIMD processor 4–8 vs 8–16 · SIMD lane/processor 2–4 vs 8–16 · SIMD thread/processor 2–4 vs 16–32 · cache ใหญ่สุด 8 MB vs 0.75 MB · main memory 8–256 GB vs 4–6 GB

สูตร รวมสูตรที่ใช้คำนวณ

Quick reference

Perf / CPU time	$Perf_X/Perf_Y = Time_Y/Time_X$ · **CPU time = IC × CPI × Cycle time** · CPI เฉลี่ย = Σ $CPI_i \times IC_i/IC$
Cache	Block addr = floor(addr/B) · index = Block addr mod #blocks · offset = $\log_2$B · index = $\log_2$#blocks · **AMAT = hit + miss rate × penalty**
Disk / Pipeline	Access = seek + **0.5/(rpm/60)** + size/rate + controller · T_c = stage ช้าสุด · เวลา = (n+k−1)×T_c
Data rep	2's: $-x = \sim x+1$ · ช่วง $-2^{n-1}...2^{n-1}-1$ · SM/1's: $\pm(2^{n-1}-1)$ · IEEE: $(-1)^S \times 1.m \times 2^{E-127}$
RISC-V / อื่น ๆ	shift i บิต = $\times/\div 2^i$ · branch target = PC + imm×2 · array offset = index×8 · Availability = MTTF/(MTTF+MTTR)

Ch04 ท้ายบท: Dependability · Virtual Memory

อยู่ในสไลด์ (ติวยังไม่ได้พูด)

- **Dependability**: Reliability = **MTTF** · ซ่อม = **MTTR** · MTBF = MTTF + MTTR · **Availability = MTTF/(MTTF+MTTR)** (เพิ่มโดย ↑MTTF หรือ ↓MTTR)
- **Hamming distance** (จำนวนบิตที่ต่าง): 2 = **ตรวจ** error 1 บิต (parity) · 3 = **แก้** 1 บิต + ตรวจ 2 บิต · parity bit อยู่ตำแหน่งกำลังของ 2 · **SEC/DEC** (distance 4) · **ECC DRAM** = 8 บิต ต่อทุก 64 บิต
- **Virtual memory**: main memory = "cache" ของ disk · virtual → physical · block = **page** (≈4 KB) · miss = **page fault** (ช้ามาก, OS จัดการ) · **page table** (index ด้วย virtual page no.; มี physical page no. + referenced/dirty bit) · **fully associative** · **write-back** เท่านั้น · **TLB** = cache ของ page table (16–512 PTE, hit 0.5–1 cycle, miss rate 0.01–1%)
- **Cache coherence** (หลาย core): **snooping** (ดัก bus; invalidate ก่อนเขียน) vs **directory-based** · **Virtual machine**: host/VMM จำลอง guest OS

ทบทวนด่วน ข้อหลอก / โจทย์ที่อาจารย์ถามกลับในคลาส

★ "คำถามสไตล์ประมาณแบบเนี้ย ไปทบทวนเนาะ"

① **SISD** = 1 คำสั่ง 1 ข้อมูล (ไม่ใช่หลายคำสั่งพร้อมกัน)
② **Pipeline** ใช้ได้ทั้ง RISC และ CISC · ลำดับ F → D → E → WB
③ **Exponent**: single = **8** บิต · double = **11** บิต
④ จุมากสุดในพีระมิด = **Offline / Magnetic tape** (ถูกสุดด้วย)
⑤ **RAM** = ชั่วคราว · **ROM** = ถาวร · **SRAM** เร็วกว่า · **DRAM** ถูกกว่า
⑥ miss น้อยสุด = **Fully associative** · เขียน cache: Write-through / Write-back / Write-allocate
⑦ **Daisy chain** = priority · ตัวเลือกว่าใครได้ใช้ bus = **Bus Arbitrator** (ไม่ใช่ Control Unit/control line)
⑧ I/O เร็วสุด = **DMA** (แทรกได้ทุก cycle) · การ์ดจอเข้าเมนบอร์ดผ่าน **PCIe**
⑨ **Track** = วงแหวน · **Sector** = ส่วนย่อยของ track · **DVD** density > CD
⑩ **InfiniBand** = ความเร็วสูงสุด (GPU↔memory/server) · **SSD** เร็ว/ประหยัดไฟกว่า แต่แพงกว่า
⑪ **Shift ซ้าย** = ×2 · **ขวา** = ÷2 · `ADD A,B,C` → A = B + C · **LOAD** Mem→Reg · **STORE** Reg→Mem
⑫ **2's complement**: ลบ = invert + 1 · ช่วง 8 บิต −128...+127 · Sign-Mag มี 2 ศูนย์
⑬ **Little-endian** = LSB ที่ address ต่ำสุด · **Big** = MSB ที่ address ต่ำสุด
⑭ **Cache**: ดู Index → เทียบ Tag → เช็ก Valid · block ใหญ่ ≠ miss น้อยเสมอ
⑮ **CPU time** = IC × CPI × Cycle time · "เร็วกว่า" = เวลาน้อยกว่า

ตัวเลขที่ต้องจำ: Arbitration **4** · I/O module **5** หน้าที่ · I/O command **4** · เทคนิค I/O **3** · Device ID **4** · RAID **0–6** · Pipeline **5** stage / **3** hazard · Flynn **4** · opcode **7** บิต · reg **5** บิต (32 ตัว) · instruction **32** บิต · Cache miss rate 1/2/4/8-way = **10.3 / 8.6 / 8.3 / 8.1 %** · IEEE bias **127**